

HEX - Business AI

Supply-chain risk & decision intelligence, built on your own ERP data

A technical overview - what it does, how it works, what it is built with and why.

SAP Hackfest 2026 - github.com/yuvraj4461/hex-business-ai

Contents

1. The problem
2. What HEX does
3. Architecture
4. Technology choices and why
5. Data model
6. Integration layer
7. The 5-agent decision graph
8. World Watch - real-time intelligence
9. Exposure engine and scenarios
10. Deterministic finance engine
11. Ask Your Data - conversational analytics
12. AI Copilot
13. Reliability - the deterministic-first principle
14. Design system
15. Security and governance
16. Deployment
17. The platform surface
18. What's next

1. The problem

Every company that moves physical goods sits on two disconnected worlds. On one side is its own operational data - suppliers, purchase orders, shipments, inventory, orders, revenue and costs - locked inside an ERP or accounting system. On the other side is a stream of external shocks - a closed shipping lane, a new tariff, an FX swing, a drought, a conflict - that lands in the news but never in a spreadsheet.

The question that matters - "how does this event actually hit us: which suppliers, which routes, how much revenue, how much cost" - normally requires a data analyst, days of work, and a fresh model each time. Most people in the business never get an answer.

- ERP data is rich but siloed and hard to query without SQL.
- External risk intelligence exists but is generic, not tied to your lanes.
- The analyst is a bottleneck for every "what does X mean for us" question.
- Spreadsheet models are stale the moment a number changes.

2. What HEX does

HEX connects to a company's ERP / accounting / commerce system, extracts the operational data, continuously watches the outside world, and runs a multi-agent analysis that translates any global event into concrete supplier, route, cost and revenue exposure for that specific business.

On top of that sits a conversational layer: anyone can ask the company's data a plain-language question and get a number, a chart and the rows behind it - no SQL, no analyst.

Core capabilities

- Integration layer - connect an ERP, a SQL replica, upload files, or use the Merge.dev unified API.
- 5-agent decision graph - finance, sales, operations, world-watch and risk agents run in sequence.
- World Watch - real-time geopolitical, disaster, FX and price intelligence, scored and matched to your corridors.
- Risk Center & Scenarios - turn any event into projected exposure and compare route decisions.
- Ask Your Data - conversational analytics over the org's own data, with persisted threads.
- Deterministic finance engine - 48 standard financial formulas, computed exactly, never by an LLM.
- AI Copilot - a chat assistant grounded in the deterministic engines and live web research.

3. Architecture

HEX is a three-tier application with a deliberately thin AI layer.

The stack, top to bottom

- Next.js 16 / React 19 frontend (Vercel) - the command-center UI.
- FastAPI backend (Render) - REST API, ~24 routers, OpenAPI-documented.
- PostgreSQL (Render) - 32 tables, org-scoped, Alembic-migrated.
- Integration layer - adapters -> raw-record landing zone -> normalizers -> canonical tables.
- LangGraph agent graph - fault-isolated, sequential specialist agents.
- World Watch pipeline - GDELT + FX + web-search collectors on a 20-minute schedule.
- Deterministic engines - app/analytics (semantic query layer) and app/finance (formula library).
- Google Gemini - used ONLY to turn questions into structured requests and to explain results in prose. It never does arithmetic or invents facts.

Request flow for a Copilot question: the API classifies the question, runs the selected agents (each pulling verified data), optionally does live web research, hands the computed findings to Gemini for a single synthesis pass, and falls back to a deterministic executive summary if Gemini is slow or unavailable.

4. Technology choices and why

Backend

- FastAPI - async, type-checked request/response models, automatic OpenAPI docs, minimal boilerplate.
- SQLAlchemy 2.0 (Mapped / mapped_column) + Alembic - typed ORM and versioned schema migrations that run automatically on deploy.
- LangGraph - a real state graph for the agent pipeline: explicit nodes, edges and shared state, with per-node fault isolation.
- APScheduler - in-process cron for auto-sync and the World Watch refresh, with no extra infrastructure.
- cryptography (Fernet) - connection credentials are encrypted at rest and never returned in an API response or a log.
- pandas / openpyxl - parse uploaded CSV / XLSX files and the World Bank commodity workbook.
- google-generativeai - the official Gemini SDK, wrapped in a lazy, resilient client.

Frontend

- Next.js 16 App Router + React 19 - route groups, typed routes, server-rendered shell.
- Tailwind CSS v4 - CSS-based config with design tokens exposed as utilities.
- Recharts 3 - themed chart wrappers (bar, area, donut) that match the design system.
- lucide-react - a single consistent icon set.

Why Gemini is kept on a short leash

- LLMs hallucinate numbers - so every figure in HEX comes from tested Python, not the model.
- The free tier returns 503 / 429 under load - so every AI call has retries, model fallback, a wall-clock budget and a deterministic fallback path.
- Google retired the 2.x models for new keys - so the client probes 3.6-flash -> flash-latest -> 2.5 -> 2.0 and caches whichever works.

5. Data model

PostgreSQL, 32 tables, every row scoped to an `organization_id`. The core entities mirror a real ERP:

- Commercial - customers, products, orders, order_items, transactions, expenses, services.
- Supply chain - suppliers, supply_routes, purchase_orders (+ lines), shipments, inventory, locations, product_materials.
- Intelligence - global_events, market_signals, agriculture_signals, commodity_forecasts, demand_signals/forecasts, business_exposures.
- Platform - organizations, users, connections, raw_records, data_threads (+ messages), scenarios, recommendations, audit_logs.

Provenance is first-class: a `SourceTrackedMixin` adds `source_connection_id`, `source_external_id` and `synced_at` to every ingested entity, so any number can be traced back to the connection and external record it came from.

6. Integration layer

The layer that turns HEX from a demo into a product. It is source-agnostic by design.

Components

- SourceAdapter ABC + registry - one class per source type (file upload, SQL read-replica, Merge.dev accounting API).
- raw_records landing zone - every fetched record is stored verbatim with a content hash, so re-syncs are idempotent.
- Per-entity normalizers - map a source's columns to HEX's canonical schema using a per-connection mapping, defaulting to sensible column names.
- run_sync(connection) - fetch -> land -> normalize -> upsert -> advance cursor, with per-record savepoints; it writes an audit row and never raises.
- Fernet-encrypted credentials - stored in connections.credentials_encrypted; the API only ever exposes a has_credentials boolean.
- Automated sync - APScheduler polls each connection on its own interval; inbound webhooks (HMAC-verified) trigger an immediate sync.
- Data-readiness scoring - per-domain coverage and freshness so the user knows which analyses are trustworthy.

7. The 5-agent decision graph

Built with LangGraph. Execution is sequential and deliberate: finance -> sales -> operations -> world-watch -> risk. Each later agent reads the findings the earlier ones produced, and risk runs last so it can weigh everything.

Fault isolation

- Every agent is wrapped in a node that catches exceptions, records the failure in `agent_runs`, and lets the graph continue.
- A partial answer from four agents beats a 500 error in a live demo.

The agents

- Finance Agent - runs the deterministic finance engine (section 10) and raises threshold recommendations: thin margin, revenue contraction, volatile revenue, short runway, weak LTV:CAC, below break-even.
- Sales Agent - order volume, fulfilment rate, cancellation rate, revenue trend, best / worst products.
- Operations Agent - reads supply capacity against demand, flags bottlenecks.
- World Watch Agent - matches recent high-severity events and FX shocks to the org's active corridors.
- Risk Agent - synthesises exposure: affected routes, revenue at risk, cost impact, an overall risk score.

In the Copilot the user can narrow "Consult" to any subset of the five; the graph and the grounding both respect that choice.

8. World Watch - real-time intelligence

A continuous pipeline that keeps the outside-world picture current.

Collectors

- GDELT DOC 2.0 API - geopolitical and natural-disaster news, classified by type and scored for severity (no more "UNKNOWN").
- Frankfurter FX API - USD / EUR / CNY against INR; a move over 2% raises an FX_SHOCK market signal.
- SerpApi / Tavily web search - standing queries for tariffs, price shocks, freight and inflation; provider auto-detected from the key format.

Orchestration

- `run_world_watch(db)` runs all collectors, prunes stale noise, then recomputes exposure for every org against new HIGH / CRITICAL events.
- Scheduled every 20 minutes by APScheduler and by a GitHub Actions cron that also keeps the free Render instance warm.
- Surfaced as a live feed, an incident-activity chart and a scrolling headline ticker.

9. Exposure engine and scenarios

The bridge between an event and a balance sheet.

- `geo_exposure.event_affects()` - a corridor keyword matcher (Red Sea, Suez, Hormuz, Malacca, Panama, Taiwan, Black Sea, Air) plus lane-to-corridor inference.
- `recompute_exposure()` - rebuilds `business_exposures` from open shipments on affected lanes, with a route-level fallback for lanes with no shipment yet.
- Wired lazily into the global-exposure summary and exposed as an explicit recompute endpoint.
- Per-event scenario analysis - any event, not just the demo Red Sea one, produces a full assessment: exposure, route alternatives (Cape of Good Hope, Air Freight), financial trade-off and an AI recommendation, with a human approve / reject step.

When an event does not intersect the org's lanes, HEX says so explicitly rather than showing empty zero cards.

10. Deterministic finance engine

The user asked for a Finance Agent that is fast and trustable. The answer was to remove the LLM from the maths entirely.

app/finance/formulas.py - 48 pure functions

- Time value of money - FV, PV, CAGR, NPV, IRR (bisection), payback, loan payment.
- Profitability - gross / operating / net margin, ROI, ROAS, markup, contribution margin.
- Liquidity & solvency - current / quick ratio, working capital, debt-to-equity, DSCR, interest coverage.
- Cash flow - free cash flow, burn rate, runway, cash-conversion cycle.
- Unit economics - CAC, LTV, LTV:CAC, CAC payback, churn, retention.
- Growth - growth rate, geometric-mean growth, moving average, least-squares forecast.
- Risk & statistics - stdev, coefficient of variation, volatility, Sharpe, Sortino, max drawdown, z-score, beta.
- Operations finance - break-even, inventory turnover, DIO / DSO / DPO, EOQ, reorder point, GMROI.
- Property & markets - cap rate, cash-on-cash, dividend yield, P/E, PEG.

How it is used

- Every function raises a clean error on undefined input instead of returning NaN.
- app/finance/engine.py runs a battery against the org's real data and returns each metric together with the formula string and the inputs used - fully auditable.
- A calculator API (GET /finance/formulas, POST /finance/calc) exposes the whole library.
- The /finance page shows the computed battery and a live calculator.
- Honest about limits: HEX has no balance sheet, so "cash position" is a labelled cumulative-cash-flow proxy and contribution margin uses operating margin.

11. Ask Your Data - conversational analytics

A Hex.tech-style "ask your data" surface, scoped to what the ERP data supports.

- Semantic layer - a fixed vocabulary of 9 metrics x ~10 dimensions (time, product, category, supplier, country, status).
- Deterministic-first planner - unambiguous questions ("revenue by month", "top 5 products") never touch the LLM, so they answer in ~85 ms even when Gemini is rate-limited; only vague phrasing pays the model round-trip.
- Safe executor - builds parameterised SQLAlchemy, never raw SQL, always filtered by organization_id, with a hard row cap.
- Narrator - a deterministic sentence by default; optional Gemini phrasing behind a flag.
- Threads - conversations persist in the database, so follow-ups ("break it down by category", "only 2026") build on the previous query and reload later.

12. AI Copilot

A chat assistant for the whole business.

- Multi-turn memory - the last several turns are sent so HEX resolves "why?" and "what about last year?".
- Web-grounded, selectively - it only does live web research when the question is actually about the outside world; an internal question no longer pulls a loosely-matched Wikipedia article.
- Agent selection - the user chooses which specialist agents to consult; grounding respects the choice.
- Off-topic guard - a query with no business / market / risk meaning gets a short "here is what I can help with" reply instead of a fabricated analysis.
- Deterministic core - every figure comes from the engines; Gemini only writes the prose, and a clean deterministic executive summary is served if it is unavailable.

13. Reliability - the deterministic-first principle

The single design decision that shapes the whole system: an LLM is a translator and an explainer, never a source of truth or a calculator.

- All arithmetic is tested Python (test_finance.py, test_analytics_ask.py, test_analytics_overview.py, test_world_watch.py, test_exposure_real.py, test_ingestion.py).
- Every Gemini call has: retry-on-503, model fallback, a wall-clock budget, and a deterministic fallback answer.
- Every analytical answer degrades gracefully - a slower, plainer, still-correct result instead of an error.
- The system states its data limits rather than guessing past them.

14. Design system

A dark "command center" instrument-panel theme.

- Violet brand accent (#7c6df5) on a deep-navy ground; colour tokens exposed as Tailwind utilities.
- Severity is meaningful - four tones (critical / elevated / stable / live) mapped from any backend status string, never hand-picked.
- Panel - the signature surface, with a 3px severity rail on its leading edge. IntelCard - a viewport-bounded, internally-scrolling panel with a subtle 3D hover.
- All numbers render mono + tabular with Indian-format currency helpers.
- Navigation is data-driven from a single source, feeding both the sidebar and a full-width "Platform" mega-menu.

15. Security and governance

- JWT authentication; an auth guard on the frontend prevents any app-shell flash before redirect.
- Six roles (Super Admin, Data Admin, Analyst, Business User, Decision Maker, External Partner) with explicit permission sets; every endpoint declares the permission it needs.
- Fernet encryption for connection credentials at rest.
- Append-only audit_logs - every mutating action writes a row; a dedicated Audit Log page renders it.
- Every database query is scoped to the caller's organization_id.

16. Deployment

- Backend - Render, native Python 3.13, with PostgreSQL. The app self-migrates on startup (alembic upgrade head in the lifespan), so a deploy needs no manual DB step.
- Frontend - Vercel, root directory frontend, API URL baked at build time, auto-deploys on push to main.
- World Watch - a GitHub Actions cron hits the refresh endpoint every 20 minutes with a shared token.
- render.yaml at the repo root recreates the whole environment.
- ~40 commits; CORS allows the Vercel preview domains via regex.

17. The platform surface

Fourteen authenticated pages under a single app shell:

- Command Center - business, operations and global intelligence on one panel.
- Global Intelligence - live disruption feed, markets, agriculture, FX.
- Risk Center - turn an event into supplier / route / financial exposure.
- Analytics - a 5-section performance dashboard (financial, sales, customer, product, operations).
- Ask Your Data - conversational analytics with charts.
- Finance - the deterministic engine: computed metrics and a formula calculator.
- Supply Routes - lanes, corridors and in-transit shipments.
- Integrations - connect an ERP or upload files; data-readiness.
- Scenarios - simulate a disruption and compare route decisions.
- Agents - the 5-agent graph, readiness and a live run.
- AI Copilot - ask HEX anything, grounded in your data.
- Approvals / Audit Log - human decisions and the append-only record.

18. What's next

- Merge.dev Link onboarding - one-click connect to 50+ accounting / commerce systems.
- Agentic analytics notebook - SQL / Python cells an agent can write and run.
- Context Studio - a governance layer for business definitions and semantic models.
- More ERP adapters (Tally, SAP), multi-currency, cross-source entity resolution.
- Row-level-security tenant isolation for true multi-tenant SaaS.